Title of the Project

Retrieval Augmented Generation (RAG) Application Using Pinecone and GPT-2

Description of the Project

This project implements a **Retrieval Augmented Generation (RAG)** system that allows users to upload a text-based Q&A document and ask questions related to its content. Instead of relying only on a language model's internal knowledge, the system first retrieves the most relevant information from the uploaded document and then generates an answer using that retrieved context.

The application uses **SentenceTransformer** to convert document text into embeddings and stores them in **Pinecone**, a vector database. When a user asks a question, the system retrieves the most relevant document chunks using similarity search and passes them as context to a **GPT-2 text generation model**. The entire application is built using **Streamlit**, providing a simple and interactive web interface.

Tools and Libraries Used

Tool / Library	Purpose
Python	Core programming language
Streamlit	Web application interface
SentenceTransformers	Text embedding generation
Transformers (GPT-2)	Answer generation
Pinecone	Vector database for retrieval
NumPy	Numerical processing
VS Code	Development environment

Code Implementation (with Explanation)

---

1. Importing Required Libraries

import streamlit as st

```
from sentence_transformers import SentenceTransformer

from transformers import pipeline

from pinecone import Pinecone, ServerlessSpec
```

Explanation:
These libraries are used to build the web interface, generate embeddings, generate answers using GPT-2, and store vectors in Pinecone.

---

2. Loading Embedding and Generation Models

```
@st.cache_resource

def load_models():

    embed_model = SentenceTransformer("all-MiniLM-L6-v2")

    generator = pipeline("text-generation", model="gpt2")

    return embed_model, generator
```

Explanation:

- The **SentenceTransformer** model converts text into numerical embeddings.
- The **GPT-2 model** generates answers based on retrieved context.
- Caching improves performance by loading models only once.

---

3. Loading and Preparing Documents

```
@st.cache_data

def load_documents(uploaded_file):

    text = uploaded_file.read().decode("utf-8")

    lines = [line.strip() for line in text.split("\n") if line.strip()]

    documents = [

        f"{lines[i]} {lines[i+1]}"

        for i in range(0, len(lines) - 1, 2)

    ]

    return documents
```

Explanation:
The uploaded text file is read and split into question–answer pairs, which are treated as individual documents for embedding and storage.

---

4. Pinecone Vector Database Setup

```
def init_pinecone():
    pc = Pinecone(api_key=PINECONE_API_KEY)
    if INDEX_NAME not in [idx["name"] for idx in pc.list_indexes()]:
        pc.create_index(
            name=INDEX_NAME,
            dimension=384,
            metric="cosine",
            spec=ServerlessSpec(cloud="aws", region="us-east-1")
        )
    return pc.Index(INDEX_NAME)
```

Explanation:
Pinecone is initialized and a vector index is created to store document embeddings for fast similarity search.

---

5. Storing Embeddings

```
def store_embeddings(index, documents, embed_model):
    embeddings = embed_model.encode(documents).tolist()
    index.upsert([
        {
            "id": str(i),
            "values": embeddings[i],
            "metadata": {"text": documents[i]}
        }
        for i in range(len(documents))
    ])
```

Explanation:
Each document is converted into an embedding and stored in Pinecone along with its original text.

---

6. Retrieving Relevant Context

```
def retrieve_context(query, embed_model, index, top_k=2):
    query_vector = embed_model.encode(query).tolist()
    result = index.query(vector=query_vector, top_k=top_k, include_metadata=True)
    return "\n".join(match["metadata"]["text"] for match in result["matches"])
```

Explanation:
When a user asks a question, the system retrieves the most relevant document chunks using cosine similarity.

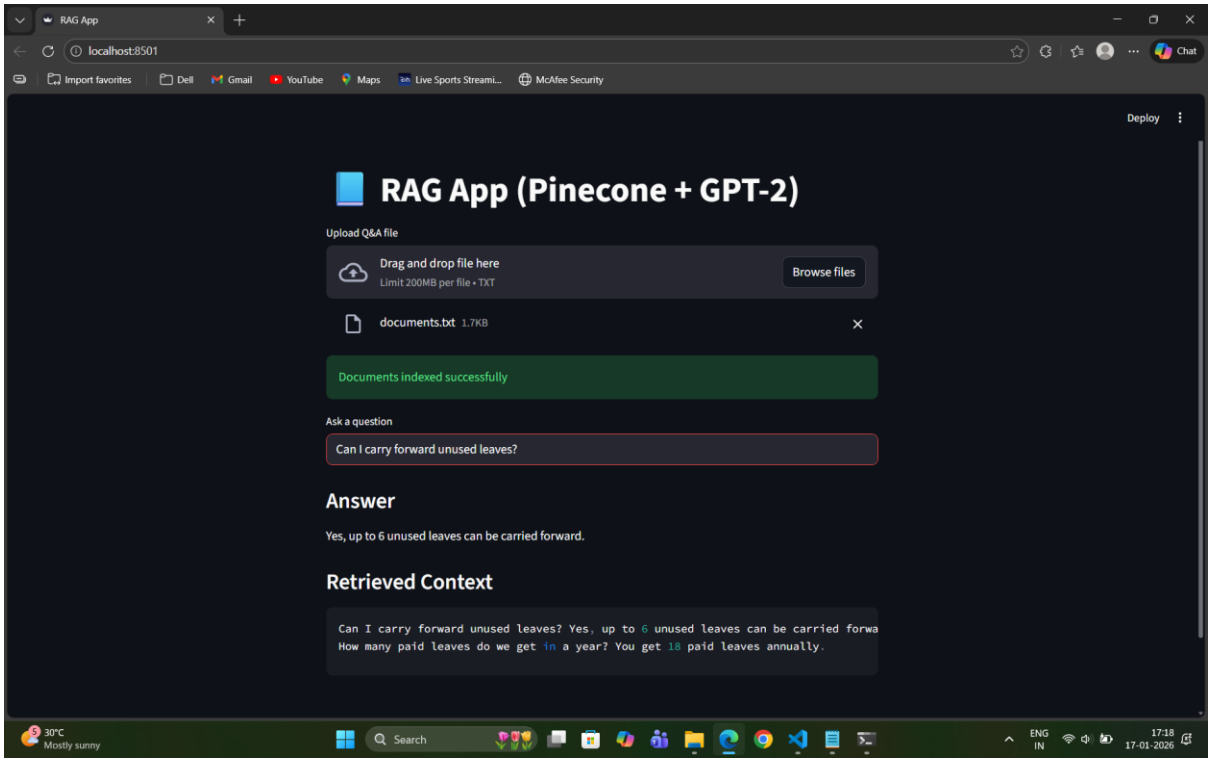
---

7. Generating the Answer

```
def generate_answer(query, context, generator):
    prompt = f"""
Use ONLY the information given in the context below.
If the answer is not present, say "I don't know."

Context:
{context}

Question:
{query}

Answer:
"""
    output = generator(prompt, max_new_tokens=160, temperature=0.2)
    return output[0]["generated_text"].split("Answer:")[-1].strip()
```

Explanation:
The retrieved context is combined with the user's question and passed to GPT-2 to generate an accurate answer.

Expected Output

- User uploads a **TXT Q&A file**

- Document embeddings are stored in Pinecone
- User enters a question
- Relevant context is retrieved from Pinecone
- GPT-2 generates an answer based on retrieved context
- The answer and retrieved context are displayed on the screen